

Informe Técnico — Sistema de Recuperación de Información

Materia: Recuperación de Información

Docente: Prof. Iván Carrera

Proyecto: 1er Bimestre

Fecha de entrega: 01 de junio de 2026

Resumen

Se diseñó e implementó un motor de búsqueda de ofertas laborales orientado a egresados de la Escuela Politécnica Nacional (EPN). El sistema indexa **47 322 documentos** mediante un índice invertido, implementa tres modelos clásicos de recuperación (Jaccard, Coseno TF-IDF y BM25) y un modelo semántico basado en embeddings con ChromaDB. Los resultados se evalúan con Precision@K, Recall@K y MAP sobre 24 consultas de prueba derivadas de las carreras de la EPN.

1. Descripción del Corpus

El corpus fue construido a partir de archivos CSV de ofertas laborales recopiladas de plataformas de empleo y organizadas por carrera de la EPN. Cada subcarpeteta de `data/raw/` contiene un archivo `*_Merged.csv` con las ofertas asociadas a una carrera específica.

Atributo	Descripción
<code>job_id</code>	Identificador único de la oferta
<code>job_title</code>	Título del puesto
<code>description_final</code>	Descripción completa del puesto
<code>careers_required</code>	Carrera primaria asociada
<code>all_careers</code>	Lista de todas las carreras que etiquetan la oferta

Una misma oferta puede aparecer en múltiples CSVs (si es relevante para varias carreras). Durante la construcción del corpus se agrupan todas las etiquetas de carrera por `job_id` antes de deduplicar, preservando esa información en `all_careers` — columna que se usa luego para construir los *qrels* de evaluación.

```
In [ ]: import sys, ast
from pathlib import Path
import pandas as pd

BASE_DIR = Path("..")
sys.path.insert(0, str(BASE_DIR / "src"))

df = pd.read_csv(BASE_DIR / "data" / "processed" / "corpus_limpio.csv").fillna("")

# Contar documentos relevantes por carrera
conteo_carreras = {}
for _, fila in df.iterrows():
    try:
        carreras = ast.literal_eval(fila["all_careers"])
    except Exception:
        carreras = []
    for c in carreras:
        conteo_carreras[c] = conteo_carreras.get(c, 0) + 1

print(f"Total de documentos únicos : {len(df):,}")
print(f"Categorías de carrera      : {len(conteo_carreras)}")
print(f"Longitud media descripción : {df['description_final'].str.split().str.len().mean():,}")
print()
print("Documentos relevantes por carrera (top 10):")
for carrera, n in sorted(conteo_carreras.items(), key=lambda x: -x[1]):
    print(f" {carrera:<40} {n:>6,}")
```

2. Decisiones de Diseño

2.1 Pipeline de Preprocesamiento

El texto pasa por dos etapas antes de ser indexado:

Etapas 1 — Normalización (preprocessing.py):

Convierte a minúsculas, elimina tildes mediante normalización Unicode NFD y remueve caracteres especiales. El resultado es texto limpio con solo letras, números y espacios.

Etapas 2 — Tokenización NLP (indexer.py):

Usando NLTK se aplica *stemming* con `SnowballStemmer("spanish")` y se filtran las *stopwords* en español. Se descartan tokens de longitud ≤ 1 . Este mismo pipeline se aplica a las consultas en tiempo de búsqueda, garantizando consistencia.

2.2 Índice Invertido

Estructura: `indice[término][doc_id] = frecuencia`. Adicionalmente se almacena `longitud_docs[doc_id]` (número de tokens por documento) para normalización en TF-IDF y BM25. El índice se persiste en disco como `pickle` (~24 MB) y se carga una sola vez al iniciar el sistema, compartiéndose entre los tres modelos clásicos.

El campo indexado concatena `job_title + description_final + careers_required`, dando mayor cobertura léxica a cada documento.

2.3 Modelos de Recuperación

| Modelo | Fórmula de score | Características | |---|---|---| | **Jaccard** | $J(Q,D) = \frac{|Q \cap D|}{|Q \cup D|}$ | Binario, ignora frecuencias | | **Coseno TF-IDF** | $\text{score}(q,d) = \frac{\text{vec}\{q\} \cdot \text{vec}\{d\}}{\|\text{vec}\{q\}\| \cdot \|\text{vec}\{d\}\|}$ | Pondera rareza de términos con IDF | | **BM25** | $\sum_t \text{IDF}(t) \cdot \frac{f(t,d)(k_1+1)}{f(t,d)+k_1(1-b+b \frac{|d|}{\text{avgdl}})}$ | Satura TF, normaliza por longitud | | **Semántico** | $\text{score} = \frac{1}{1+d_{L2}}$ | Dens retrieval, captura significado |

BM25 usa los parámetros estándar: $k_1=1.2$ (saturación del TF) y $b=0.75$ (normalización por longitud).

Para BM25 e IDF de Coseno, las normas de documentos y los valores IDF se **precomputan al inicializar** el modelo, evitando cálculos repetidos en cada consulta.

2.4 Recuperación Semántica

Se usa el modelo `paraphrase-multilingual-MiniLM-L12-v2` de `sentence-transformers`, elegido por su soporte nativo para español y su tamaño reducido (117 MB). Los embeddings de los ~47 k documentos se generan offline y se almacenan en una colección `ChromaDB` persistente (`data/processed/chroma_db/`). En tiempo de consulta, solo se codifica la consulta y se busca por vecindad en el espacio vectorial.

3. Ejemplos de Consultas y Resultados

```
In [ ]: from models import cargar_indice, BuscadorJaccard, BuscadorCoseno, BuscadorBM25
        from embeddings import BuscadorSemantico
```

```
datos = cargar_indice()
jac = BuscadorJaccard(datos_indice=datos)
cos = BuscadorCoseno(datos_indice=datos)
bm25 = BuscadorBM25(datos_indice=datos)
sem = BuscadorSemantico()
```

```
In [ ]: def mostrar_comparacion(consulta: str, top_k: int = 5):
        """Muestra los top-K resultados de los 4 modelos para una misma consulta."""
        modelos = [
            ("Jaccard", jac),
            ("Coseno TF-IDF", cos),
            ("BM25", bm25),
            ("Semántico", sem),
        ]
        print(f"\n{'='*70}")
```

```

print(f"  Consulta: \"{consulta}\"")
print(f"{' '*70}")
for nombre, modelo in modelos:
    resultados = modelo.buscar(consulta, top_k=top_k)
    print(f"\n [{nombre}]")
    for i, (doc_id, score) in enumerate(resultados, 1):
        fila = df[df["job_id"] == doc_id]
        titulo = fila.iloc[0]["job_title"] if not fila.empty else doc_id
        print(f"    {i}. {score:.4f} | {titulo[:60]}")

# Consulta 1: búsqueda técnica con términos precisos
mostrar_comparacion("desarrollador software backend python django microservicios")

# Consulta 2: búsqueda con términos de carrera
mostrar_comparacion("ingeniero civil construccion puentes infraestructura vial")

# Consulta 3: consulta semántica – sin términos literales del corpus
mostrar_comparacion("persona que maneje equipos grandes y tome decisiones estratégicas")

```

Observaciones sobre los ejemplos:

- **Consulta 1 (técnica):** Los modelos léxicos (BM25, Coseno) rankean bien porque los términos `python`, `django`, `backend` aparecen literalmente en las descripciones.
- **Consulta 2 (de carrera):** Los cuatro modelos muestran resultados similares; el corpus tiene muchas ofertas etiquetadas explícitamente con `ingenieria civil`.
- **Consulta 3 (semántica):** El modelo de embeddings recupera puestos como *Gerente de Proyectos* o *Director de Operaciones* que comparten el **concepto** aunque no las palabras exactas. Los modelos léxicos fallan aquí al no encontrar coincidencias directas con `maneje`, `decisiones` o `estratégicas` en las formas raíz del corpus.

4. Evaluación de Métricas

4.1 Construcción de Qrels

Se definen 24 consultas de prueba, una por carrera de la EPN. Los documentos relevantes para cada consulta son todos los `job_id` que tienen esa carrera en su columna `all_careers`. Este enfoque aprovecha la estructura existente del corpus sin requerir anotación manual.

Métricas calculadas:

- $P@K = \frac{|\text{relevantes} \cap \text{top-K}|}{K}$ — precisión en los primeros K resultados
- $R@K = \frac{|\text{relevantes} \cap \text{top-K}|}{|\text{relevantes}|}$ — cobertura de los relevantes en top-K
- $AP = \frac{1}{|\text{rel}|} \sum_{k: d_k \in \text{rel}} P@k$ — precisión promedio por consulta

- $MAP = \frac{1}{|Q|} \sum_{q} AP(q)$ — promedio de AP sobre todas las consultas

```
In [ ]: from evaluation import construir_qrels, evaluar_modelo

print("Construyendo qrels...")
qrels = construir_qrels(df)
print(f"Qrels: {len(qrels)} consultas | "
      f"promedio {sum(len(v) for v in qrels.values())/len(qrels):.0f} docs relevant")

modelos_eval = {
    "Jaccard": jac,
    "Coseno TF-IDF": cos,
    "BM25": bm25,
    "Semántico": sem,
}

resultados_k10 = {}
resultados_k10000 = {}

for nombre, modelo in modelos_eval.items():
    print(f" Evaluando {nombre}...")
    resultados_k10[nombre] = evaluar_modelo(modelo, qrels, k=10)
    resultados_k10000[nombre] = evaluar_modelo(modelo, qrels, k=10000)
```

```
In [ ]: print("\n=== k = 10 (primeros 10 resultados) ===")
print(f"{'Modelo':<18} {'MAP':>8} {'P@10':>8} {'R@10':>8}")
print("-" * 46)
for nombre, rep in resultados_k10.items():
    print(f"{'nombre':<18} {rep['MAP']:>8.4f} {rep['P@10_mean']:>8.4f} {rep['R@10_mean']:>8.4f}")

print("\n=== k = 10 000 (ranking completo) ===")
print(f"{'Modelo':<18} {'MAP':>8} {'P@10k':>8} {'R@10k':>8}")
print("-" * 46)
for nombre, rep in resultados_k10000.items():
    print(f"{'nombre':<18} {rep['MAP']:>8.4f} {rep['P@10000_mean']:>8.4f} {rep['R@10000_mean']:>8.4f}")
```

4.2 Análisis de Resultados

Los resultados obtenidos fueron los siguientes:

k = 10 (calidad del top de resultados):

Modelo	MAP	P@10	R@10
Jaccard	0.0015	0.4625	0.0020
Coseno TF-IDF	0.0046	0.5250	0.0060
BM25	0.0025	0.5458	0.0036
Semántico	0.0011	0.3417	0.0016

k = 10 000 (calidad del ordenamiento global):

Modelo	MAP	P@10k	R@10k
Jaccard	0.2243	0.1528	0.5571
Coseno TF-IDF	0.2643	0.1522	0.5554
BM25	0.2564	0.1578	0.5714
Semántico	0.1214	0.0868	0.3595

Interpretación:

- **BM25 lidera en Precision y Recall globales** (k=10000): su normalización por longitud de documento evita que ofertas extensas dominen el ranking, logrando el mayor R@10k (0.5714) y la mayor P@10k (0.1578).
- **Coseno TF-IDF tiene el mejor MAP** en ambos valores de k: indica que los documentos relevantes aparecen en posiciones más altas del ranking en promedio. El peso IDF premia términos raros y específicos, lo que mejora la calidad del orden.
- **Jaccard es competitivo** a pesar de ser el modelo más simple. Al ignorar frecuencias y trabajar con conjuntos binarios, su rendimiento es razonablemente cercano al de los modelos vectoriales.
- **El modelo semántico obtiene las métricas más bajas** en esta evaluación (MAP=0.1214 vs 0.2643 de Coseno). Esto se explica por la naturaleza de los *qrels*: los documentos relevantes se definen por coincidencia de carrera, y las consultas de prueba contienen los **términos exactos** de cada carrera. Los modelos léxicos son naturalmente favorecidos en este escenario. Sin embargo, el semántico supera a los modelos léxicos en consultas expresadas en lenguaje coloquial o con paráfrasis (como se observa en la consulta 3 de la sección anterior).
- El Recall es bajo para k=10 en todos los modelos (máximo 0.006). Esto es **esperado y no preocupante**: con ~1 500 documentos relevantes por carrera en promedio, solo 10 resultados no pueden cubrir más del 0.6%. La métrica relevante para la experiencia del usuario es P@10.

5. Comparación de Modelos — Cuándo usar cada uno

Escenario de consulta	Modelo recomendado	Razón
Términos técnicos precisos (<code>"python django REST"</code>)	BM25	Maneja bien TF saturado; no sobrepondera docs largos

Escenario de consulta	Modelo recomendado	Razón
Búsqueda por carrera ("ingeniería civil")	Coseno TF-IDF	IDF premia términos específicos de carrera
Lenguaje coloquial / paráfrasis	Semántico	Captura significado sin depender de coincidencia léxica
Corpus grande, consultas cortas	BM25	Robusto y eficiente en la práctica
Prototipo rápido / sin NLP	Jaccard	Simple, sin necesidad de normalización de vectores

Casos donde semántico mejora respecto a BM25/TF-IDF:

- Consultas con sinónimos: "liderazgo de equipos" → recupera puestos con "gestión de personas"
- Consultas en otro idioma: el modelo multilingüe permite buscar en inglés y encontrar ofertas en español
- Consultas descriptivas: "alguien que trabaje con números y tome decisiones financieras" → Economista / Analista Financiero

Casos donde semántico empeora:

- Consultas con términos técnicos muy específicos ("BIM Revit AutoCAD") donde la coincidencia exacta es crítica
- Consultas muy cortas (1-2 palabras) donde el embedding no tiene suficiente contexto

6. Conclusiones

Se implementó un sistema de recuperación de información completo con cuatro modelos sobre un corpus de 47 322 ofertas laborales. Los principales hallazgos son:

1. **BM25 y Coseno TF-IDF son los modelos más robustos** para este corpus y tipo de consultas, con MAP~0.26 en ranking completo.
2. **Jaccard, siendo el modelo más simple**, muestra un desempeño sorprendentemente cercano a los modelos vectoriales (MAP=0.2243), lo que sugiere que la simple coincidencia de términos ya captura bien la relevancia en este dominio.
3. **El modelo semántico complementa** a los clásicos en escenarios de lenguaje natural, aunque las métricas formales (basadas en etiquetas de carrera) no reflejan su ventaja real en consultas libres.
4. La **construcción automática de qrels** desde `all_careers` es una estrategia práctica cuando no se cuenta con juicios de relevancia manuales, aunque introduce un sesgo hacia modelos léxicos.